

Credit Card Fraud Detection using Machine Learning

Project Overview

In this project, we are working on detecting fraudulent credit card transactions using machine learning. Fraud detection is crucial in the financial industry as it helps to prevent financial losses and maintain customer trust. The dataset we are using comes from Kaggle and contains transaction data where each record is either fraudulent (1) or legitimate (0).

We will use Logistic Regression, a machine learning algorithm, to predict whether a transaction is fraudulent or legitimate. To handle the imbalance in the data, we will apply the SMOTE technique to generate synthetic data for the minority class (fraudulent transactions).

Dataset Information

The dataset has the following key characteristics:

- **Features:** The dataset contains various features like Time, V1, V2, ..., V28, and Amount, which describe different aspects of a credit card transaction.
- **Target Variable:** The target variable is Class, where 1 indicates a fraudulent transaction and 0 indicates a legitimate transaction.
- **Class Imbalance:** The dataset is highly imbalanced, with far more legitimate transactions than fraudulent ones. This is common in fraud detection datasets.

1. Loading and Exploring the Dataset

We begin by loading the dataset and exploring its structure to understand its features and check for missing values.

```
import pandas as pd
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score, recall_score, precision_score,
accuracy_score
```

```
# Load the dataset
df = pd.read_csv(r'C:/Users/HP PC/Desktop/credit/creditcard.csv')
```

```

# Display basic information about the dataset
print(df.info()) # Info gives the column names and data types
print("First ten values:\n", df.head(10)) # Display the first ten rows
print("\nStatistical Values of the dataset:\n", df.describe()) # Basic statistics like mean, std, etc.

# Check for missing values
print("\nDataset With missing values from Row 1-10:\n", df.iloc[0:10].isnull())
print(df.isnull().sum()) # Total number of missing values in each column

```

Explanation:

df.info(): Provides a summary of the dataset including the number of records, data types of each column, and whether any columns have null values.

df.head(10): Displays the first 10 rows of the dataset to give an overview of its structure.

df.describe(): Shows statistical summaries for numerical columns (e.g., mean, standard deviation).

df.isnull().sum(): Checks the dataset for missing values.

2. Handling Class Imbalance with SMOTE

Since the dataset is imbalanced, where fraudulent transactions are much less frequent, we apply SMOTE (Synthetic Minority Over-sampling Technique) to balance the dataset by generating synthetic examples of the minority class.

```

# Checking the distribution of classes
class_counts = df['Class'].value_counts() # Count of fraudulent and legitimate transactions
print("\nClass Distribution:\n ", class_counts)

# Separating features (X) and target variable (y)
X = df.drop(columns=['Class']) # Features
y = df['Class'] # Target

# Applying SMOTE to balance the dataset
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

print(f'Class distribution after SMOTE:\n{y_resampled.value_counts()}') # New class distribution

```

Explanation:

`df['Class'].value_counts()`: Shows the number of fraudulent (1) and legitimate (0) transactions in the original dataset.

SMOTE: Balances the dataset by creating synthetic examples for the minority class (fraudulent transactions).

3. Splitting the Data into Training and Test Sets

Next, we split the data into training and testing sets. This helps to evaluate the model's performance on unseen data.

```
# Splitting the dataset into training and testing sets  
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.3,  
random_state=42)
```

```
# Check the shapes of the training and testing sets  
print(f"Training set size: {X_train.shape}, {y_train.shape}")  
print(f"Testing set size: {X_test.shape}, {y_test.shape}")
```

Explanation:

`train_test_split()`: Splits the resampled data into training (70%) and testing (30%) sets. The `random_state=42` ensures that the split is reproducible.

We print the shapes of the training and test sets to confirm the split.

4. Training the Logistic Regression Model

We train a Logistic Regression model on the training data. Logistic Regression is a widely used method for binary classification tasks like fraud detection.

```
# Train Logistic Regression Model  
model = LogisticRegression(max_iter=1000) # Logistic Regression with max iterations set to  
1000  
model.fit(X_train, y_train) # Fit the model on the training data
```

Explanation:

`LogisticRegression(max_iter=1000)`: Initializes the logistic regression model. The `max_iter=1000` ensures that the model has enough iterations to converge.

`model.fit()`: Trains the model using the training data (`X_train` and `y_train`).

5. Model Evaluation

We evaluate the trained model using several performance metrics: Precision, Recall, F1-Score, and Accuracy. These metrics are crucial, especially when dealing with imbalanced data like fraud detection.

```
# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model using precision, recall, f1-score, and accuracy
print("Classification Report:")
print(classification_report(y_test, y_pred)) # Detailed classification report

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
fsc = f1_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1-Score: {fsc}")
print(f"Accuracy: {accuracy}")
print("Number Of Transactions Predicted as Fraudulent:", sum(y_pred))
```

Explanation:

`classification_report()`: Provides a detailed report containing metrics such as precision, recall, and F1-score for both classes (fraudulent and legitimate).

Precision: The fraction of relevant instances among the retrieved instances (how many predicted frauds were actually fraud).

Recall: The fraction of relevant instances that have been retrieved (how many actual frauds were correctly identified).

F1-Score: The harmonic mean of precision and recall.

Accuracy: The overall accuracy of the model.

Finally, we print the number of fraudulent transactions predicted by the model.

NOTES:

The model provides useful insights into detecting fraudulent credit card transactions. From the classification report, we can assess how well the model is performing. Depending on the results, we may consider improving the model by:

Trying more complex models like Random Forests or XGBoost.

Tuning the model's hyperparameters.

Further feature engineering, such as scaling numerical features or handling categorical features differently.

Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Visualizing the class distribution
sns.countplot(x='Class', data=df)
plt.title('Class Distribution')
plt.show()
```

This will give a bar plot showing the number of fraudulent vs. legitimate transactions in the dataset.